

AED2 2026 (1s) - AP-08 -ORDENAÇÃO EM MEMÓRIA EXTERNA

Instruções:

1. E/S: tanto a entrada quanto a saída de dados devem ser "secas", ou seja, sem frases explicativas. Siga o modelo fornecido e apenas complete as partes informadas (veja os exemplos fornecidos);
2. Identificadores de variáveis: escolha nomes apropriados;
3. Documentação: inclua cabeçalho, comentários e indentação no programa;
4. O código-fonte deve ser desenvolvido em linguagem C;
5. Não é permitido o uso da função `qsort/sort(...)` da linguagem C/C++ ou o uso de estruturas do tipo `<list>` ou `<vector>`;
6. Tempo limite de solução no sistema Judge: 1 segundo;
7. Submeta o programa no sistema Judge (link igual para ambas as turmas): <https://judge.unifesp.br/AEDII1S2026IN/>;
8. Plágios serão checados pela plataforma MOSS (<https://theory.stanford.edu/~aiken/moss/>). Caso detectados, não serão tolerados, sendo a submissão descartada.
9. Soluções que não sigam as diretrizes acima serão desconsideradas.

Descrição

Você recebeu a tarefa de ordenar um grande volume de dados. No entanto, o sistema possui restrições severas de memória principal. Para resolver isso, você deve implementar um sistema de Ordenação em Memória Externa dividido em duas fases: a geração de blocos iniciais via Seleção por Substituição e a fusão desses blocos via Intercalação Balanceada (*K-Way*).

Como estamos trabalhando em um ambiente de *Online Judge*, não utilizaremos arquivos físicos no disco. Em vez disso, vamos emular as fitas magnéticas (ou *streams* de dados). O acesso a esses dados deve ser estritamente sequencial, simulando ponteiros de leitura e escrita de um arquivo (*fread/fwrite*).

O limite de memória do *Judge* estará configurado para um valor baixo. Tentativas de alocar um vetor dinâmico de tamanho N na memória interna para burlar o processo podem resultar em erro do tipo *Memory Limit Exceeded*.

Estrutura e Ordenação

- Memória Interna/principal (Fase 1): Será usado um *Min-Heap* de tamanho máximo M . Cada posição armazena apenas um valor numérico. Ele gerenciará os elementos ativos e os elementos marcados para o próximo bloco.
- Memória Interna/principal (Fase 2): Um *Min-Heap* de tamanho máximo K . Cada nó deste *heap* armazena o valor do elemento e o *id_da_fita/stream* de onde ele veio.
- Fitas Magnéticas/*Streams* Emuladas: Devem-se emular $2 * K$ fitas/*streams*, em que as primeiras K fitas são o primeiro conjunto de canais e as K fitas restantes são o segundo conjunto.

Formato de Entrada e Saída

Entrada

A entrada consiste em 2 linhas contendo números inteiros:

- A primeira linha contém três inteiros: N (número total de elementos), M (capacidade do *Heap* na Fase 1) e K (número de fitas de entrada/saída na intercalação).
- A segunda linha contém N inteiros desordenados separados por espaço.

Saída

Para garantir a implementação correta das estruturas de dados, o seu programa deve exibir o estado interno dos *Heaps* e o resultado das fitas exatamente nos momentos descritos abaixo:

1. **[Fase 1 - *Heap* Inicial]:** Exiba o vetor do *Heap* de tamanho M logo após ler os primeiros M elementos e executar o *Build-Heap*.
2. **[Fase 1 - *Heap* Alterado]:** Exiba o vetor do *Heap* de tamanho M no exato momento em que o primeiro bloco terminar de ser escrito (quando o contador de elementos ativos zerar e os elementos do "próximo bloco" tomarem conta do *Heap*).
3. **[Fase 1 - Fitas Geradas]:** Exiba o conteúdo apenas das fitas utilizadas para a escrita (índices 0 a $K - 1$) após o término da Seleção por Substituição. Indique os valores presentes nos blocos entre colchetes $[N_1 N_2 \dots]$.
4. **[Fase 2 - *Heap* de Intercalação]:** No início da Fase 2, das intercalações, logo após ler o primeiro elemento de cada uma das K fitas ativas e montar o *Heap* de Intercalação, exiba os valores contidos nesse *Heap*.
5. **[Fase 2 - Fitas após 1ª Passada]:** Exiba o estado apenas das fitas de destino (índices K a $2K - 1$) imediatamente após a conclusão da primeira passada completa de intercalação. Isso significa que todos os blocos gerados na Fase 1 devem ter sido lidos, mesclados e gravados no segundo conjunto de fitas.
6. **[Resultado Final]:** Exiba a sequência completamente ordenada obtida ao final do processo.

Exemplos de casos de Teste

Exemplo 01: Cenário Padrão

Entrada:

```
12 3 2
20 15 4 5 12 1 10 11 32 8 7 2
```

Saída:

```
[Fase 1 - Heap Inicial]: 4 15 20
[Fase 1 - Heap Alterado]: 11 10 1
[Fase 1 - Fitas Geradas]:
Fita 0: [4 5 12 15 20] [2 7 8]
Fita 1: [1 10 11 32]
[Fase 2 - Heap Intercalacao]: 1 4
[Fase 2 - Fitas apos 1a Passada]:
Fita 2: [1 4 5 10 11 12 15 20 32]
Fita 3: [2 7 8]
[Resultado Final]: 1 2 4 5 7 8 10 11 12 15 20 32
```

Exemplo 02: Vetor Inversamente Ordenado (força a criação de muitos blocos pequenos).

Entrada:

```
8 3 2
80 70 60 50 40 30 20 10
```

Saída:

```
[Fase 1 - Heap Inicial]: 60 70 80
[Fase 1 - Heap Alterado]: 30 40 50
[Fase 1 - Fitas Geradas]:
Fita 0: [60 70 80] [10 20]
Fita 1: [30 40 50]
[Fase 2 - Heap Intercalacao]: 30 60
[Fase 2 - Fitas apos 1a Passada]:
Fita 2: [30 40 50 60 70 80]
Fita 3: [10 20]
[Resultado Final]: 10 20 30 40 50 60 70 80
```

Exemplo 03: Elementos Repetidos e *Heap* Menor

Entrada:

```
9 2 2
5 5 2 2 9 9 1 1 5
```

Saída:

```
[Fase 1 - Heap Inicial]: 5 5
[Fase 1 - Heap Alterado]: 2 2
```

```
[Fase 1 - Fitas Geradas]:  
Fita 0: [5 5] [1 1 5]  
Fita 1: [2 2 9 9]  
[Fase 2 - Heap Intercalacao]: 2 5  
[Fase 2 - Fitas apos 1a Passada]:  
Fita 2: [2 2 5 5 9 9]  
Fita 3: [1 1 5]  
[Resultado Final]: 1 1 2 2 5 5 5 9 9
```

Exemplo 04: Vetor Já Ordenado (Gera apenas 1 bloco grande na Fita 0)

Entrada:

```
6 3 2  
10 20 30 40 50 60
```

Saída:

```
[Fase 1 - Heap Inicial]: 10 20 30  
[Fase 1 - Fitas Geradas]:  
Fita 0: [10 20 30 40 50 60]  
Fita 1:  
[Fase 2 - Heap Intercalacao]: 10  
[Fase 2 - Fitas apos 1a Passada]:  
Fita 2: [10 20 30 40 50 60]  
Fita 3:  
[Resultado Final]: 10 20 30 40 50 60
```